CSE145 Capstone Milestone M1

Part 1:

Name: Arshad Ali ERP ID: 33448

1.1 De Morgan laws

Law 1: (xy)' = x' + y'

x	y	xy	(xy)'	x'	y'	x'+y'
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Columns (xy)' and x'+y' are the same in all rows, so law 1 is true.

Law 2: (x+y)' = x'y'

x	y	x+y	(x+y)'	x'	y'	x'y'
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Columns (x+y)' and x'y' are the same, so law 2 is true.

1.2 Counting

(a) N variables give 2^N rows in the truth table. Every row can have output 0 or 1, so the number of functions is 2^(2^N).

- N = 2: 2^4 = 16
- N = 3: 2^8 = 256
- N = 4: 2^16 = 65,536

Five of the 16 functions for N = 2: AND, OR, XOR, NAND, NOR (also XNOR, constant 0, constant 1, x, y, x', y', x -> y ...)

(b) An n-bit adder has two n-bit inputs, so 2n input bits and 2^(2n) rows (no carry-in counted).

- 4-bit: 2^8 = 256 rows
- 8-bit: 2^16 = 65,536 rows
- 16-bit: 2^32 = 4,294,967,296 rows

1.3 Personal function F(x,y,z)

ERP ID 33448, last three digits 448, so D = 448.

B = 448 mod 256 = 448 - 256 = 192

192 = 128 + 64 = 11000000 (b7…b0)

192 is not in {0, 255, 23, 105}, so no change.

x	y	z	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

F = x'y'z' + x'y'z

Number of product terms = 2 (one for each 1 in the output column).

1.4 Half adder and full adder

Half adder

x	y	S	C	x XOR y	x AND y
0	0	0	0	0	0
0	1	1	0	1	0
1	0	1	0	1	0
1	1	0	1	0	1

S = XOR column, C = AND column.

Full adder

x	y	cin	S	cout	ODD	MAJ
0	0	0	0	0	0	0
0	0	1	1	0	1	0
0	1	0	1	0	1	0
0	1	1	0	1	0	1
1	0	0	1	0	1	0
1	0	1	0	1	0	1
1	1	0	0	1	0	1
1	1	1	1	1	1	1

S = ODD (1 when an odd number of inputs are 1), cout = MAJ (1 when at least two inputs are 1).

Sum of products:

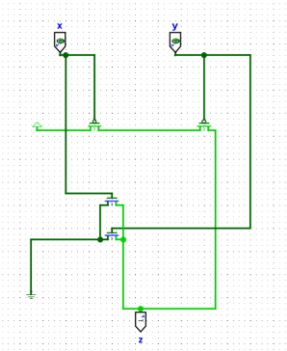
S = x'y'cin + x'y cin' + xy'cin' + xy cin

cout = x'y cin + xy'cin + xy cin' + xy cin

PART 2:

2a. NOT_T:

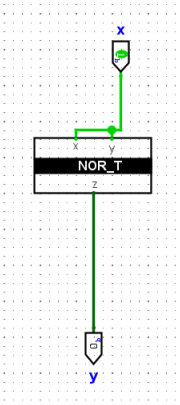
Top part = pull-up network (2 PMOS in series). Bottom part = pull-down network (2 NMOS in parallel).



X	Y	P1	P2	N1	N2	Z
0	0	on	on	off	off	1
0	1	on	off	off	on	0
1	0	off	on	on	off	0
1	1	off	off	on	on	0

2b. NOT_NOR

Y = X'. Idea: NOR(X, X) = (X + X)' = X'. One NOR_T with X on both inputs.

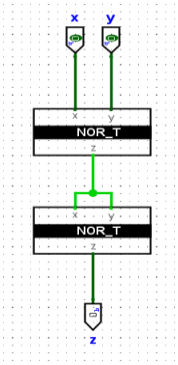


X	Y
0	1
1	0

NOR_T gates: 1, transistors: 4.

2c. OR_NOR

Z = X + Y. Idea: OR is NOR followed by NOT. NOR_T #1 takes X and Y, and its output goes to both inputs of NOR_T #2 (works as a NOT).

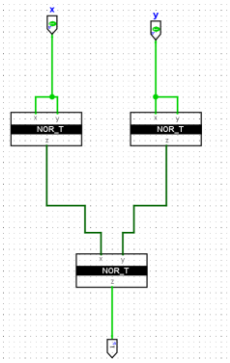


X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

NOR_T gates: 2, transistors: 8.

2d. AND_NOR

Z = XY. Idea (De Morgan): XY = (X' + Y')'. Two NOT_NOR make X' and Y', then one NOR_T combines them.



X	Y	X'	Y'	Z	
0	0	1	1	0	
0	1	1	0	0	
1	0	0	1	0	
1	1	0	0	1	

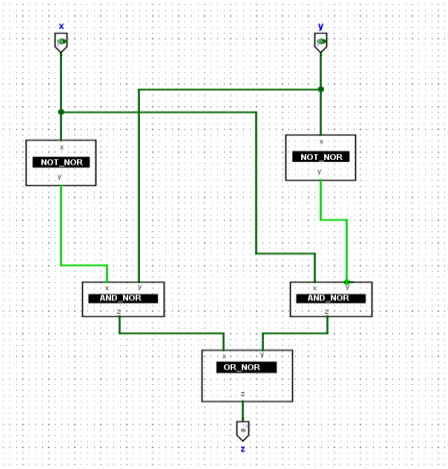
NOR_T gates: 3 (1 + 1 + 1), transistors: 12.

2e. XOR_SOP

Z = X'Y + XY'. Built with 2 NOT_NOR (X' and Y'), 2 AND_NOR (X'Y and XY'), and 1 OR_NOR.

X	Y	X'	Y'	X'Y	XY'	Z
0	0	1	1	0	0	0
0	1	1	0	1	0	1
1	0	0	1	0	1	1
1	1	0	0	0	0	0

Gate count after expanding: 2 NOT_NOR x 1 + 2 AND_NOR x 3 + 1 OR_NOR x 2 = 10 NOR_T gates, so 40 transistors.



SUMMARY:

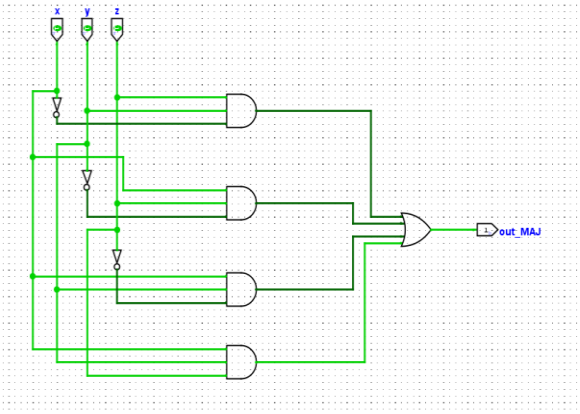
Subcircuit	NOR_T gates	Transistors
NOT_NOR	1	4
OR_NOR	2	8
AND_NOR	3	12
XOR_SOP	10	40

Part 3: Three-input circuits (sum of products)

Method (slide 36): 1) name the variables, 2) write the truth table, 3) find the rows where the output is 1, 4) make one AND for each of those rows and OR the results.

3a. MAJ3

X	Y	Z	OUT
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

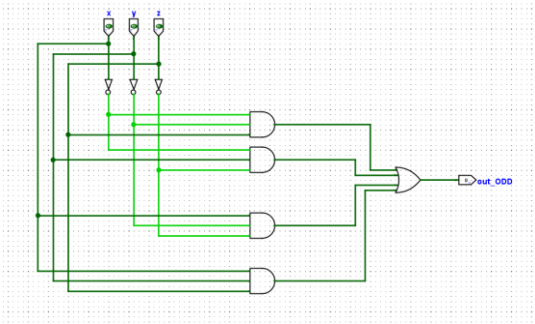


Rows with 1: 011, 101, 110, 111.

OUT = X'YZ + XY'Z + XYZ' + XYZ (4 AND gates, 1 OR gate)

3b. ODD3

X	Y	Z	OUT
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



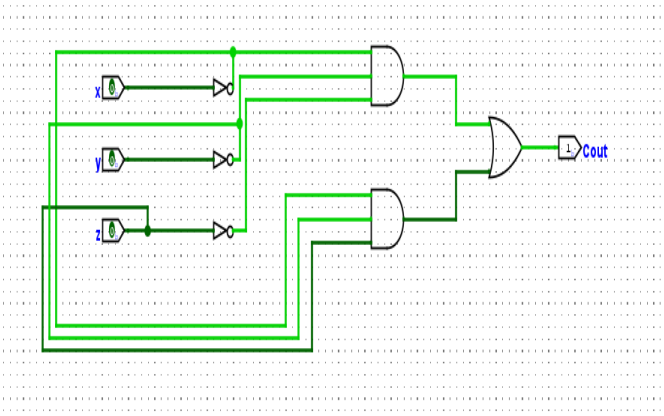
Rows with 1: 001, 010, 100, 111.

OUT = X'Y'Z + X'YZ' + XY'Z' + XYZ (4 AND gates, 1 OR gate)

3c. PERSONAL_F

ERP ID 33448, D = 448, B = 448 mod 256 = 192 = 11000000. Not in {0, 255, 23, 105}, so F = 1 only for rows 000 and 001.

X	Y	Z	OUT
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0



OUT = X'Y'Z' + X'Y'Z (2 AND gates, 1 OR gate)

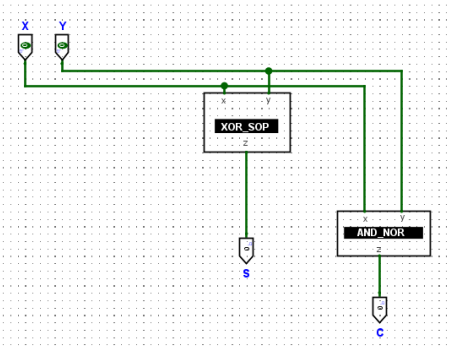
Remark: the two terms combine to X'Y', so Z does not change the output.

Part 4: Adders

4a. HALF_ADDER

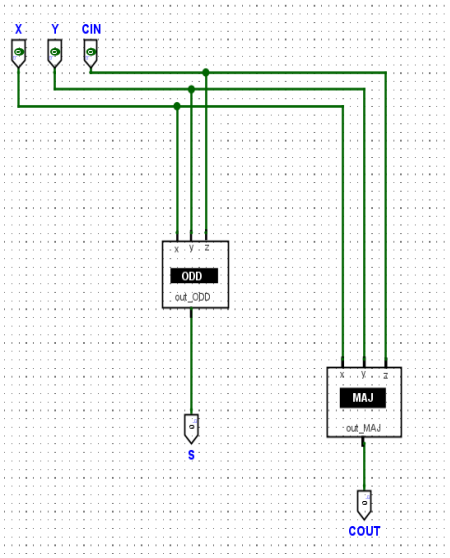
S comes from my XOR_SOP(X, Y). C comes from an AND gate of X and Y.

X	Y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



4b. FULL_ADDER_A

Built only from MAJ3 and ODD3 (slide 45): S = ODD3(X, Y, CIN) and COUT = MAJ3(X, Y, CIN), both with the same three inputs.

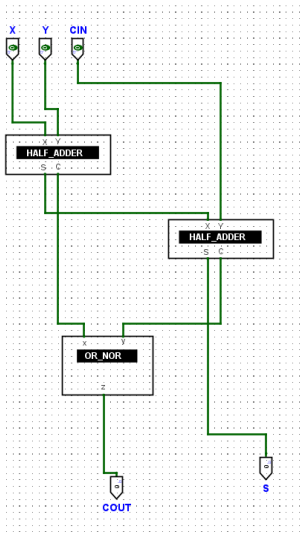


X	Y	CIN	S	COUT
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

4c. FULL_ADDER_B

HA1 adds X and Y and gives S1 and C1. HA2 adds S1 and CIN and gives the final S and C2. An OR gate combines C1 and C2 into COUT.

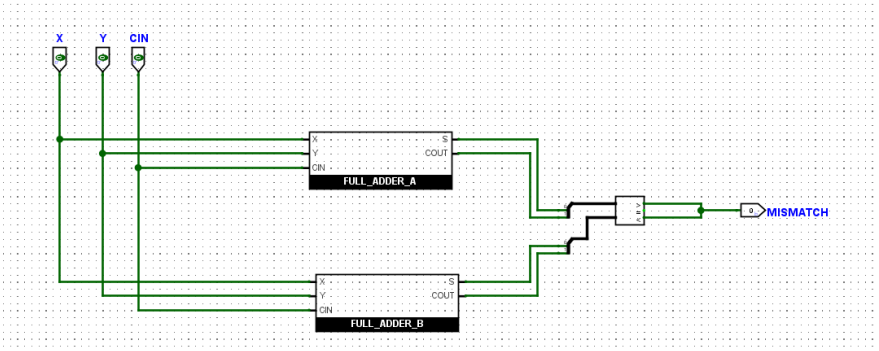
X	Y	CIN	S1	C1	C2	S	COUT
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	1	0	0	1	0
0	1	1	1	0	1	0	1
1	0	0	1	0	0	1	0
1	0	1	1	0	1	0	1
1	1	0	0	1	0	0	1
1	1	1	0	1	0	1	1



Why a plain OR is enough to combine the carries: C1 is 1 only when X = Y = 1, and then S1 = X XOR Y = 0. C2 = S1 AND CIN, so C2 is 0 whenever C1 is 1. The two partial carries can therefore never both be 1, so OR gives the correct carry out (also, X + Y + CIN is at most 3, so the carry out is at most 1).

TESTBENCH

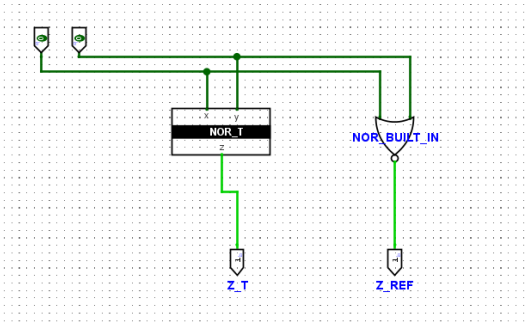
Both full adders (FA1 and FA2) are driven from the same pins X, Y, CIN. Outputs S1, COUT1, S2, COUT2. MISMATCH = (S1 XOR S2) OR (COUT1 XOR COUT2), so it is 1 whenever the two adders disagree.



X	Y	CIN	S1	COUT1	S2	COUT2	MISMATCH	2*Cout + S	X + Y + CIN
0	0	0	0	0	0	0	0	0	0
0	0	1	1	0	1	0	0	1	1
0	1	0	1	0	1	0	0	1	1
0	1	1	0	1	0	1	0	2	2
1	0	0	1	0	1	0	0	1	1
1	0	1	0	1	0	1	0	2	2
1	1	0	0	1	0	1	0	2	2
1	1	1	1	1	1	1	0	3	3

MISMATCH is 0 in all 8 rows, and 2*Cout + S equals X + Y + CIN in every row.

NOR_T checked against the built-in NOR:



X	Y	NOR_T	Built-in NOR	Agree ?
0	0	1	1	yes
0	1	0	0	yes
1	0	0	0	yes
1	1	0	0	yes

Gate count and delay comparison

What I counted as one gate: each built-in AND, OR or NOT counts as 1 gate whatever its number of inputs, Negate bubbles are free, and each NOR_T counts as 1 gate (not split

into transistors). The longest path is the largest number of gates a signal passes through from an input to an output.

- MAJ3 and ODD3: 3 NOT + 4 AND + 1 OR = 8 gates each, longest path NOT -> AND -> OR = 3 gates.
- XOR_SOP: 10 NOR_T, longest path NOT_NOR (1) + AND_NOR (2) + OR_NOR (2) = 5 gates.
- HALF_ADDER: 10 NOR_T + 1 AND = 11 gates.

	Primitive gates	Longest path (gates)
FULL_ADDER_A	16 (8 + 8)	3
FULL_ADDER_B	23 (2 x 11 + 1 OR)	10

For FULL_ADDER_B the longest path is the sum output: XOR in HA1 (5 gates) then XOR in HA2 (5 gates). The carry path is shorter.

Building everything from NOR_T

- XOR_SOP: 10 NOR_T = 40 transistors (from Part 2).
- FULL_ADDER_B: each half adder is XOR_SOP (10) + AND_NOR (3) = 13 NOR_T, so two are 26, plus OR_NOR (2) = 28 NOR_T = 112 transistors.
- FULL_ADDER_A (unoptimized SOP): 3 NOT_NOR for X', Y', CIN' (3 NOR_T). Each 3-input AND = 2 AND_NOR = 6 NOR_T, so 4 of them = 24. Each 4-input OR = 3 OR_NOR = 6 NOR_T. That is 30 NOR_T per function, 60 for both, plus 3 = about 63 NOR_T = 252 transistors.

Does universal mean efficient? No. Universal only means NOR can build any function, not that it builds it cheaply. A plain AND needs 3 NOR_T (12 transistors and 2 gate delays) when a direct AND needs far fewer, and XOR_SOP costs 40 transistors and 5 gate delays. Using one gate type is simple, but it costs more area, power and delay than using the right gate for each job.

PART 5: Reflection and Presentation:

R1. Each row with output 1 gives an AND term that is true only for that row. OR-ing all of them gives exactly the truth table, so it works for any function. The drawback is size: n inputs can need up to 2^n terms, so the circuit gets very big.

R2. A 16-bit adder has 32 inputs, so the table would need 2^32 (about 4.3 billion) rows. Instead we use modularity: build one 1-bit full adder and chain 16 of them.

R3. FULL_ADDER_A is better for speed and gate count (16 gates vs 23, longest path 3 vs 10). The cost is that if everything is made from NOR_T, A needs about 63 NOR_T while B needs only 28.

R4. Transistor (a switch, hides the physics) -> NOR_T (hides the 4 transistors) -> NOT, OR, AND from NOR_T (hide how they use NORs) -> XOR_SOP (hides its gates) -> HALF_ADDER (hides XOR and AND) -> FULL_ADDER_B (hides two half adders and the OR). Each layer only shows its inputs and outputs.

R5. NOR is universal: NOT, OR and AND can all be made from NOR, so every other gate can come from it (I did this in Part 2). NAND from NOR_T: use two NOR_T to make X' and Y', a third NOR_T on them to get XY, and a fourth NOR_T to invert it. That gives (XY)' using 4 NOR_T.

R6. While building ODD3 I wired some AND inputs wrongly, so it behaved like MAJ3. I saw it when xyz = 001 gave 0 instead of 1. I rechecked all 8 test rows, traced each wire, fixed the wrong ones and tested again.